

Reviewer Pack — Minimal Order of Quiver Representations and Frege Proof Leng...

Entry 3fe87713055e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 16:52:52 UTC

Minimal Order of Quiver Representations and Frege Proof Length Correlation

Entry ID: 3fe87713055e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 16:52:52 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (specifically, quiver representations) **Field B** (complexity object): Frege Proof Complexity

Statement:

For every CNF φ with n variables, the minimal order of a quiver representation that corresponds to φ is linearly correlated with its Frege proof length $f(\varphi)$, such that $\text{MinimalOrder}(\text{quiverRep}(\varphi)) = \Theta(f(\varphi))$.

Rationale (proposer's reasoning):

Quiver representations offer a structured way to encode Boolean functions, which may reveal hidden structures in CNFs. This could potentially lead to a new understanding of the complexity of Frege proofs for SAT instances.

Taxonomy category: quiver_representation_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cbcffe244126194a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for 30 random seeds, the Pearson correlation coefficient between $\text{MinimalOrder}(\text{quiverRep}(\varphi))$ and $f(\varphi)$ is ≥ 0.7 , and no seed produces a Pearson correlation coefficient < 0.5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import combinations, product

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def tseitin_encoding(cnf):
        new_vars = {}
        new_cnf = []

        def encode_clause(clause):
            if len(clause) == 1:
                return clause[0]
            else:
                v = next(v for v in range(1, max(new_vars.values()) + 2) if v not in new_vars)
                new_vars[v] = True
                new_cnf.append([-v])
                for literal in clause:
                    new_cnf.append([literal, -v])
                return v

        def encode_clauses(clauses):
            if len(clauses) == 1:
                return encode_clause(clauses[0])
            else:
                v = next(v for v in range(1, max(new_vars.values()) + 2) if v not in new_vars)
                new_vars[v] = True
                new_cnf.append([-v])
                for clause in clauses:
                    new_cnf.append([encode_clause(clause), -v])
                return v

        def encode_formula(formula):
            if isinstance(formula, list):
                return encode_clauses(formula)
            else:
                return encode_clause(formula)
```

```

        return formula

    cnf = [encode_formula(cnf) for cnf in cnf]

    return new_vars, new_cnf

def quiver_representation(cnf):
    new_vars, new_cnf = tseitin_encoding(cnf)
    n = len(new_vars)
    quiver_rep = [[0] * n for _ in range(n)]

    for clause in new_cnf:
        if len(clause) == 1:
            u = abs(clause[0]) - 1
            quiver_rep[u][u] += 1
        else:
            for literal in clause:
                u, v = abs(literal) - 1, abs(next(lit for lit in clause if lit != literal)) - 1
                quiver_rep[u][v] += 1

    return quiver_rep

def min_order(quiver_rep):
    n = len(quiver_rep)
    visited = [False] * n
    order = 0

    def dfs(node):
        nonlocal order
        if not visited[node]:
            visited[node] = True
            for neighbor in range(n):
                if quiver_rep[node][neighbor] > 0:
                    dfs(neighbor)
            order += 1

    for node in range(n):
        dfs(node)

    return n - order

def frege_proof_length(cnf):
    # Placeholder function to compute Frege proof length
    # This is a dummy implementation and should be replaced with actual logic
    return len(cnf) * 2 # Simplified example

n = random.choice([5, 10, 15, 20, 30, 40])
cnf = [[random.choice([-1, 1]) * (i + 1) for i in range(n)] for _ in range(random.randint(5, 10))]

quiver_rep = quiver_representation(cnf)
min_order_value = min_order(quiver_rep)
frege_length = frege_proof_length(cnf)

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": float('nan'), # Placeholder for actual calculation
}

```

```

        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.randint(2, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(math.isnan(res["metric_value"]) for res in results):
        print("RESULT: INCONCLUSIVE mapping_undefined")
    else:
        mean_metric_value = sum(res["metric_value"] for res in results) / len(results)
        std_metric_value = math.sqrt(sum((res["metric_value"] - mean_metric_value) ** 2 for res in results) / len(results))
        support_fraction = sum(1 for res in results if not math.isnan(res["metric_value"]) and res["metric_value"] >= mean_metric_value - std_metric_value) / len(results)

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
        else:
            first_failing_seed = next(seed for seed, res in zip(seeds, results) if not math.isnan(res["metric_value"]) and res["metric_value"] < mean_metric_value - std_metric_value)
            print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b249b901.py", line 118, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b249b901.py", line 99, in run_trial
    quiver_rep = quiver_representation(cnf)
                  ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b249b901.py", line 57, in quiver_representation
    new_vars, new_cnf = tseitin_encoding(cnf)
                        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b249b901.py", line 52, in tseitin_encoding
    cnf = [encode_formula(cnf) for cnf in cnf]
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b249b901.py", line 48, in encode_formula
    return encode_clauses(formula)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b249b901.py", line 39, in encode_clause
    v = next(v for v in range(1, max(new_vars.values()) + 2) if v not in new_vars)
        ~~~~~^

```

ValueError: max() iterable argument is empty

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it was unable to complete the required calculations for the conjecture. As a result, we cannot confirm or falsify the conjecture based on this test. | next: Investigate and fix the crash in the test code to allow for proper computation of the Pearson correlation coefficients.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13813
2	propose	ollama_remote	glm4:latest	0	0	14258
3	preregistration	ollama_remote	glm4:latest	0	0	10003
4	novelty	ollama_remote	glm4:latest	0	0	18038
5	novelty	ollama_remote	glm4:latest	0	0	16369
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23129
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12544
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23858
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18702
10	judge	ollama_remote	glm4:latest	0	0	12136

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 162851 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/3fe87713055e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/3fe87713055e.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/3fe87713055e.tar.gz (if generated)